COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S1
DATE PERFORMED:	27/08/2026	**DATE SUBMITTED:**	27/08/2026
NAME:	Ramon Brendt D. Campo	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

Prelims skills exam [SET C]

1. PROCEDURE OUTPUT

Instructions:

- Please make sure to follow all code of conduct re: examinations and related policies.

Problem Title: The Palindrome Number Problem

Problem Definition:



A palindrome number is a number that remains the same when digits are reversed. For example, the number 12321 is a palindrome number, but 1451 is not a palindrome

number. Given a positive integer, the task is to check whether the number is palindrome or not. We want to use data structures to check whether a given string of input conforms to the given description of a palindrome. The user is then prompted whether the given input is a palindrome or not.

Your task is to use two data structures for your solution:

- Stack
- Queue

But, they must have different internal representations, either:

- Array-based
- Linked List

Example: If your stack is array-based, your queue must be implemented using a linked list (and vice-versa).

You must then use the values in the given table below to validate your code.

Palindrome?	**Output (Console)**	**Remark**
10201		
1003003001		
88		
0		
202		

Submission Requirements:

- .cpp/.h and other used source code.
- PDF file for solution compilation and analysis. This must include but are not limited to:
    - Pseudocode
    - Analysis of Pseudocode
- **Record yourself explaining how you implemented your algorithm. Maximum of 3 minutes only.**
- **You will be graded by how you presented your output.**

Stack LINKED LIST implementation

HEADER file

```cpp
#ifndef PALINDROMESTACK_H
#define PALINDROMESTACK_H

#include <iostream>

// class node that contains data and the next pointer
template <typename T>
class Node {
    public:
    T data;      // value that we need to store
    Node* next;  // pointer to the next node
};

// global variable template for the head of the stack
template <typename T>
Node<T>* head = nullptr;

// operations of the stack

// push
template <typename T>
void push(T newData) {
    Node<T>* newNode = new Node<T>;
    newNode->data = newData;
    newNode->next = head<T>;
    head<T> = newNode;
}

// pop
template <typename T>
T pop() {
    if (head<T> == nullptr) {
        std::cout << "Stack Underflow! \n";
        return T();
    }

    Node<T>* temp = head<T>;
    T tempval = temp->data;
    head<T> = head<T>->next;
    delete temp;
    return tempval;
}

// top
template<typename T>
void Top() {
    if (head<T> == nullptr) {
        std::cout << "The stack is empty\n";
```

```cpp
void Top() {
    if (head<T> == nullptr) {
        std::cout << "The stack is empty\n";
        return;
    }
    std::cout << "The current top is " << head<T>->data << std::endl;
}

// is_empty
template<typename T>
bool isEmpty() {
    return head<T> == nullptr;
}

// display all
template<typename T>
void display() {
    if (head<T> == nullptr) {
        std::cout << "The stack is empty\n";
        return;
    }

    std::cout << "Displaying the stack: \n";
    Node<T>* currentNode = head<T>;

    while (currentNode != nullptr) {
        std::cout << currentNode->data << std::endl;
        currentNode = currentNode->next;
    }
    std::cout << "\n";
}

#endif
```

MAIN FILE

Prelims_skills_exam > StackLL_palindrome > StackPrelimmain.cpp > checkPalindrome(const std::string &)

```cpp
#include <iostream>
#include <string>
#include "palindromeStack.h" // Ensure this matches your header filename

bool checkPalindrome(const std::string& input) {
    // 1. Push every character of the string onto the stack
    for (char ch : input) { //for every ch ( or the number inputed) in the input it will create a new node
        push<char>(ch); //creates a brand new stack/node that has the contents of what the user inputed
    }

    // 2. Pop characters and compare them to the original string
    for (char ch : input) {
        if (pop<char>() != ch) { //pops the current top node to check, if the current top isint the same with the
            //original it returns false
            // Clear remaining stack elements if a mismatch occurs
            while (!isEmpty<char>()) { //while the stack isint empty, pop every node in there
                pop<char>();
            }
            return false;
        }
    } //if every character appears the same, returns true
    return true;
}

int main() {

    std::string testCases[] = {
        "10201",
        "1003003001",
        "88",
        "0",
        "202"
    };

    std::cout << "Palindrome?\tOutput (Console) Remark\n";
    std::cout << "-------------------------------------------\n";

    // Loop through the fixed array directly
    for (const std::string& test : testCases) {
        std::cout << test << "\t\t";
        if (checkPalindrome(test)) {
            std::cout << "Is a Palindrome\n";
        } else {
            std::cout << "Not a Palindrome\n";
        }
    }

    return 0;
}
```

Output

```
Palindrome?     Output (Console) Remark
-------------------------------------------
10201           Is a Palindrome
1003003001              Is a Palindrome
88              Is a Palindrome
0               Is a Palindrome
202             Is a Palindrome
PS C:\Users\PC\Documents\DSA CAMPO>
```

conclusion / code analysis

This code checks the statements if they are a palindrome or not by using a stack data structure which is implemented through the linked list. The header file contains the regular operations of the stack (top, display, pop, push, isFull, isEmpty and the node constructor) the logic behind checking the statement if their a palindrome or not is created in the main file where I created a boolean function that checks if the imputed statement is a palindrome by comparing the original structure to the stack by first pushing the character of each string into the stack and then once its finished its then popped and compared if its the same as the original placement because of its unique logic of “last in first out”. The statement itself is all gathered in an array which acts as a placeholder for the statement and to display it as well.

The reason why I used a linked list implementation over an array implementation is because in my previous activities, I am more experienced in implementing a linked list instead of an array, although an array would be better here because of its quick access to data. I decided to use a linked list implementation. Plus I had a similar activity to palindrome so theres also an added experience to using linked list.

Queue Array implementation

Header file

```cpp
#ifndef PALINDROMEQUEUE_H
#define PALINDROMEQUEUE_H

#include <iostream>

template <typename T>
class queueArr {
private:
    T* q_array;
    size_t q_capacity;
    size_t q_size;
    int q_front;
    int q_back;

public:
    queueArr(size_t capacity);
    ~queueArr();

    bool empty();
    bool full();
    size_t size();
    void Clear();

    T Front();
    T Back();

    void Enqueue(T newData);
    void Dequeue();
};

// Constructor
template <typename T>
queueArr<T>::queueArr(size_t capacity) {
    q_capacity = capacity;
    q_size = 0;
    q_front = 0;
    q_back = -1;
    q_array = new T[q_capacity];
}

// Destructor
template <typename T>
queueArr<T>::~queueArr() {
    delete[] q_array;
}

// empty()
template <typename T>
bool queueArr<T>::empty() {
    return q_size == 0;
}

// full()
template <typename T>
bool queueArr<T>::full() {
    return q_size == q_capacity;
}
```

```cpp
// Clear()
template <typename T>
void queueArr<T>::Clear() {
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

// Return Front Element
template <typename T>
T queueArr<T>::Front() {
    if (empty()) {
        return T{};
    }
    return q_array[q_front];
}

// Return Back Element
template <typename T>
T queueArr<T>::Back() {
    if (empty()) {
        return T{};
    }
    return q_array[q_back];
}

// Size
template <typename T>
size_t queueArr<T>::size() {
    return q_size;
}

// Enqueue
template <typename T>
void queueArr<T>::Enqueue(T newData) {
    if (full()) {
        return;
    }
    q_back = (q_back + 1) % q_capacity;
    q_array[q_back] = newData;
    q_size++;
}

// Dequeue
template <typename T>
void queueArr<T>::Dequeue() {
    if (empty()) {
        return;
    }
    q_front = (q_front + 1) % q_capacity;
    q_size--;
}

#endif
```

MAIN.cpp

```cpp
#include <iostream>
#include <string>
#include "palindromeQueue.h"

// calculates the string length
int getLength(const std::string& str) {
    int len = 0;
    for (char c : str) {
        len++;
    }
    return len;
}

// Check palindrome using queueArr without stack
bool isPalindrome(const std::string& str) {
    int len = getLength(str); //gets the string's length
    queueArr<char> q(len);

    // Load characters into queue from left to right
    for (char ch : str) {
        q.Enqueue(ch);
    }

    // Compare queue output (Left-to-Right) with string index (Right-to-Left)
    for (int i = len - 1; i >= 0; i--) {
        if (q.Front() != str[i]) {
            return false; //if the inputed str isint equal to the input then it would stop the enqueueing and return false
        }
        q.Dequeue();
    }

    return true; // if all characters matches then it will return true
}

int main() {
    // Array with empty string sentinel
    std::string testCases[] = {
        "10201",
        "1003003001",
        "88",
        "0",
        "202",
        ""
    };

    std::cout << "Input Value\tOutput\tRemark\n";
    std::cout << "-------------------------------------------\n";

    for (int i = 0; testCases[i] != ""; i++) {
        bool result = isPalindrome(testCases[i]);
        int len = getLength(testCases[i]);

        std::cout << testCases[i]
                  << (len < 8 ? "\t\t" : "\t")
                  << (result ? "Is a Palindrome" : "Not a Palindrome")
                  << "\n";
    }

    return 0;
}
```

Output

```
Input Value     Output  Remark
----------------------------------------
10201           Is a Palindrome
1003003001      Is a Palindrome
88              Is a Palindrome
0               Is a Palindrome
202             Is a Palindrome
PS C:\Users\PC\Documents\DSA CAMPO>
```

Conclusion/code analysis

This code, similar to the stack also checks statements/numbers if they are a palindrome or not but this time I used a queue data structure that's implemented using arrays, so similar to the stack, the header file remains the same as a regular queue class and contains its regular operations like the front, back, enqueue, dequeue etc. again the main function to check the statement if its a palindrome is created in the main.cpp file where since its an array I first created a length checker, which gets the length of the string (similar to the stack operation) once it gets the length I then created the function that checks if its a palindrome or not, the getlength function is then assigned to len, after the queue class is called assign the "len" variable (the input of the user), just like the stack, the string and its separate character is enqueued into the queue but unlike the stack which is last in first out where if we pop it we could easily tell if its a palindrome or not, a queue implements the "first in, first out" logic, which is different yet similar to the stack. The logic I used to check if the statement is a palindrome or not, I used the len variable and assigned it into the last part of the queue which I then traversed by decrementing it going through the whole queue right to left. If its the same placement as it was from left to to right it would be outputted as true, if not its false. The statements, just like the stack its stored in a array and it is then presented by traversing and assigning the input with the len variable in the getlength function. Which is then separated char by char. This section is where we call the palindrome checker function and then determine it if its a palindrome or not

The reason why I used an array implementation, besides the fact that I used linked list for stack is because I only recently knew how to implement a array into a header file and luckily its displayed through a queue, so for me I picked an array for convenience, even though a linked list would be better suited here cause of it not having a fixed size and its much better traversal but in the end I sticked with what I know which is implementing it into an array

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

COMPUTER ENGINEERING

QUEZON CITY

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.